
Profiling with perf and Flamegraphs

Hands-on exercises with MadGraph5_aMC@NLO and CUDACPP

Daniele Massaro

1th October 2026

Version: dev

Abstract

In this exercise session, we will apply some of the profiling techniques discussed during the lectures to profile MadGraph5_aMC@NLO with the CUDACPP for hardware-accelerated matrix element calculation instructions. We will use Linux perf to collect various performance metrics, generate the flamegraphs, and understand the differences in between the original pure Fortran version of the code and the new version exploiting Single Instruction Multiple Data (SIMD) paradigm through CPU vector instructions. Finally, we will verify Amdahl's Law by observing the speedup of the code in the new architecture. For people having access to GPU hardware, it is possible to obtain the flamegraphs also in case of GPU offloading.

Contents

1	Background	3
2	Prerequisites	3
2.1	If you are using a/your Linux machine	4
2.1.1	Copying files from a remote machine (like LXplus)	5
2.2	If you want to use the Docker image	5
2.2.1	Running the container	5
2.2.2	What is inside the container	6
2.2.3	Verifying the setup	6
2.2.4	Extracting files from the container	6
3	Exercise 1: Installation	6
3.1	Install MadGraph	6
3.2	Install the CUDACPP plugin	7
3.3	Install the FlameGraph scripts	7
3.4	Check your machine's vectorisation support	7
4	Exercise 2: Generate a process and test the setup	7
4.1	Generate the process	8
4.2	Run a first test	8
4.3	Locate the madevent executable	8
5	Exercise 3: Prepare for profiling	9
5.1	Add debug symbols and frame pointers	9
5.1.1	Fortran compilation flags	9
5.1.2	C++ and CUDA compilation flags	9

5.2	Rebuild	10
5.3	Increase the number of events	10
6	Exercise 4: Flamegraphs	11
6.1	A brief reminder on flamegraphs	11
6.2	Profile the Fortran baseline	11
6.3	Profile the vectorised version	12
6.4	Walking the stack: DWARF-based unwinding	13
7	Exercise 5: Amdahl's law verification	13
7.1	Extract the data from the flamegraphs	13
8	Exercise 6: Hardware performance counters	14
8.1	General counters	14
8.2	Floating-point width counters	15
8.3	How to interpret the results	16
9	Summary	16
10	References and useful links	17

1 Background

MadGraph5_aMC@NLO (MG5aMC) is a Monte Carlo event generator widely used in High Energy Physics, and by the main LHC experiments. At its core, MG5aMC is a meta-code: given a certain physics process, it is able to *write* Fortran code that can be used to obtain *physical events* describing the specified process. This implies computing the corresponding scattering matrix elements, and performing the phase space integration, and event unweighting, tasks that are done by the generated program **MadEvent**.

The **CUDACPP** plugin, developed at CERN, enhances the capabilities of the code generator engine, allowing it to write C++ code that can exploit SIMD vector instructions on CPUs or be offloaded to GPUs, and which replaces the Fortran matrix element calculation. The other parts of MadEvent (phase space, unweighting, ...) stay unchanged.

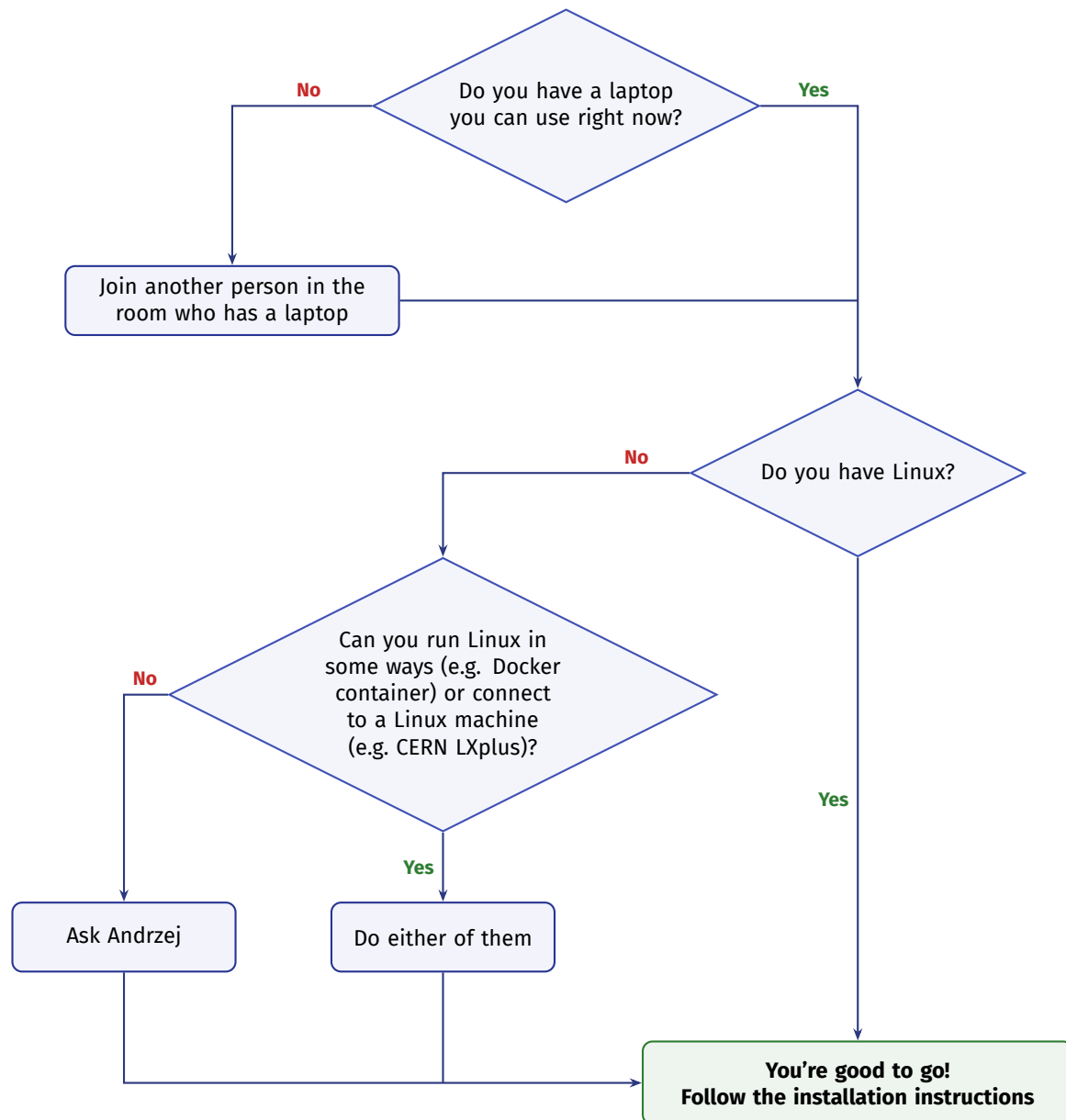
The reason on why CUDACPP tackled the matrix element computation is clear when profiling the original Fortran code.

Trivia

The name “MadGraph” comes from *Madison*, Wisconsin, where the code was originally created. It was born initially as a tool to automate Feynman diagram generation and matrix element calculation. It became a fully-fledged event generator once the component MadEvent was developed.

2 Prerequisites

Follow this decision tree to understand to which class you belong and take the necessary actions:



2.1 If you are using a/your Linux machine

If you are using LXplus or a CERN-based machine with access to CVMFS, you can get the needed software by loading the stack LCG_108 for Alma Linux 9, built with GCC 14 with frame pointers enabled:

```
source /cvmfs/sft.cern.ch/lcg/views/LCG_108/x86_64-el9-gcc14fp-opt/setup.sh
```

Check that you fulfill the following:

- Python 3.9 or more, with the `six` package
- GCC and GFortran
- GNU Make
- `perf` (if not available, install via `linux-tools-common`, `linux-tools-generic`, or `linux-tools-$(uname -r)` on Ubuntu/Debian)
- Perl (to generate FlameGraphs)

Before starting, check that perf works:

```
perf stat ls
```

2.1.1 Copying files from a remote machine (like LXplus)

If you are working on a remote machine (e.g. LXplus via SSH), the generated flamegraph SVGs live on that machine, not on your computer. To view them in a browser, you need to copy them back. Use `scp` from a terminal *on your local machine* (this works on Linux, macOS, and Windows with PowerShell or WSL):

```
scp <username>@lxplus.cern.ch:/path/to/flamegraph.svg .
```

You can also copy multiple files at once:

```
scp <username>@lxplus.cern.ch:/path/to/'*.svg' .
```

Then open the `.svg` files in your browser locally.

2.2 If you want to use the Docker image

First, verify you have Docker installed in your Linux distribution, see the [official instructions](#).

A Docker image is provided with MadGraph, the CUDACPP plugin, the FlameGraph tools, and perf pre-installed.

```
docker pull ghcr.io/qubit01/madgraph-profiling-exercises:latest
```

macOS / Apple Silicon users

The Docker image works natively on Apple Silicon without x86 emulation. CUDACPP supports ARM NEON through the `cppsse4` backend, so **Mac users using the Docker image should use `cppsse4` as the backend when running MadGraph.**

2.2.1 Running the container

The tool `perf` accesses hardware performance counters through the host kernel, so the container must run with elevated privileges: There are two options.

Option A: `--privileged` (simplest)

```
docker run -it --rm \
  --privileged \
  --pid=host \
  ghcr.io/qubit01/madgraph-profiling-exercises:latest
```

Option B: fine-grained capabilities (more restrictive)

```
docker run -it --rm \
  --cap-add SYS_ADMIN \
  --cap-add SYS_PTRACE \
  --security-opt seccomp=unconfined \
  --pid=host \
  ghcr.io/qubit01/madgraph-profiling-exercises:latest
```

Host-side prerequisites

The **host machine** must allow unprivileged access to performance counters. You can check that the following options have been set (before running the container): `sysctl kernel.perf_event_paranoid` should return `-1`; If not, you can try to set it

```
sudo sysctl kernel.perf_event_paranoid=-1
```

2.2.2 What is inside the container

Component	Location
MadGraph5	~/MadGraph5/
CUDACPP plugin	~/MadGraph5/PLUGIN/
FlameGraph scripts	/opt/FlameGraph/ (also in PATH)
perf	/usr/bin/local/perf

2.2.3 Verifying the setup

Once inside the container, run the provided check script:

```
./check_perf.sh
```

This tests that `perf stat` and `perf record` work correctly. If any test fails, verify that you started the container with the correct privileges and that the host-side `sysctl` settings are in place.

2.2.4 Extracting files from the container

Flamegraph SVGs and other outputs can be copied to the host while the container is running:

```
# From the host:
docker cp <container_id>:/home/user/path/to/flamegraph.svg .
```

Alternatively, mount a host directory as a volume when starting the container:

```
docker run -it --rm --privileged --pid=host \
-v $(pwd)/output:/home/user/output \
ghcr.io/qubitol/madgraph-profiling-exercises:latest
```

Files placed in `~/output/` inside the container will then appear in the `output/` directory on the host.

3 Exercise 1: Installation

3.1 Install MadGraph

```
wget https://github.com/mg5amcnlo/mg5amcnlo/archive/refs/tags/v3.6.6.tar.gz
tar xzf v3.6.6.tar.gz
mv mg5amcnlo-3.6.6 MadGraph5
cd MadGraph5
```

Every subsequent command assumes you are inside the MadGraph5 directory. The code can be run by running the following

```
./bin/mg5_aMC
```

3.2 Install the CUDACPP plugin

The CUDACPP plugin provides the hardware-accelerated matrix element implementations. Install it through the MadGraph CLI:

```
./bin/mg5_aMC <<EOF
install cudacpp
EOF
```

Or, more simply, open MadGraph, run `install cudacpp`, and then wait for it to finish. The plugin will be placed in the `PLUGIN/` directory.

3.3 Install the FlameGraph scripts

We will use Brendan Gregg's FlameGraph tools to convert `perf` data into Flamegraphs:

```
cd ..
git clone https://github.com/brendangregg/FlameGraph.git
export FLAMEGRAPH_DIR=$(pwd)/FlameGraph
cd MadGraph5
```

3.4 Check your machine's vectorisation support

Before choosing a CUDACPP backend to run, check the compiler flags by reading `/proc/cpuinfo`, or by running `lscpu`, you should find that at least `sse4_2` and `avx/avx2` are supported.

```
grep -o -E 'sse4_2|avx2|avx512f|avx512bw|avx512vl' /proc/cpuinfo
```

The output tells you which backends you can use:

CPU flag(s)	MadGraph Backend	Width	Doubles/instruction
sse4_2	cppsse4	128-bit	2
avx2	cppavx2	256-bit	4
avx512f or avx512bw or avx512vl	cpp512y or cpp512z	512-bit	8

Exercise 1

Install MadGraph, the CUDACPP plugin, and the FlameGraph tools. Verify that `perf` works. Check which SIMD instruction sets your machine supports and decide which backend you will use in the following exercises (whenever you find the placeholder `<backend>`, the value of the parameter `cudacpp_backend`). Specify here the backend:

4 Exercise 2: Generate a process and test the setup

4.1 Generate the process

We will use the process $gg \rightarrow t\bar{t}gg$ (gluon annihilation into a top–antitop pair with two additional gluons) as our benchmark. This process has many Feynman diagrams, making the matrix element calculation computationally expensive and therefore an interesting profiling target.

To generate this process, we will use the MadGraph command-line interface. Start MadGraph with the following command (this is the command you will always need to use in case you will need to run MadGraph):

```
./bin/mg5_aMC
```

And run the following commands, one at a time:

```
generate g g > t t~ g g
output madevent_simd MY_PROCESS
```

The first command will generate all the possible feynman diagrams, while the second one will *write the code* needed to perform the event generation. You will find out a new folder named MY_PROCESS can be found in the current working directory.

Output modes

The output mode determines which backends are available:

- `madevent_simd`: enables CPU backends (`cppnone`, `cppsse4`, `cppavx2`, `cpp512y`, `cpp512z`)
- `madevent_gpu`: enables GPU backends (`cuda`, `hip`)

4.2 Run a first test

Now we will run the generated code, using the `launch` command. Restart MadGraph if you closed it, and run the following commands (replace `<backend>` with the appropriate backend for your machine, see § 3.4):

```
launch MY_PROCESS
done
set cudacpp_backend <backend>
set nevents 10000
done
```

The `done` command in the middle serves for the purpose of skipping one of the prompts, while the one at the end is used to start the event generation. The `set` command can be used only in the second prompt, when the program is asking to modify `param_card.dat` or `run_card.dat`.

If this completes without errors and produces a cross-section result, your setup is working.

4.3 Locate the madevent executable

MadGraph creates one `madevent` executable per subprocess. The various subprocesses are contained in the `SubProcesses` directory, in this case we have only one single subprocess directory (`P1_gg_ttxgg`):

```
cd MY_PROCESS/SubProcesses/P1_gg_ttxgg/
ls -la madevent
```

The `madevent` file is a symlink to the actual compiled binary (e.g. `madevent_cpp` for the vectorised backend, and `madevent_cuda` or `madevent_hip` for the GPU backend). The first launch also creates subdirectories `G1/`, `G2/`, ..., that represent the various *integration channels of the phase space*. The thing you need to know is that each one of them contains a text file, named `input_app.txt`, with the parameters for that specific phase-space parametrisation.

When using `launch`, MadGraph orchestrates the run of the executable `madevent` on the different parametrisations determined by the `G*/` folders for a certain subprocess. This means `madevent` represents the *smallest* component of the full computation. It is the script that it is worth profiling.

Test running `madevent` directly, it requires standard input that can be passed through the `input_app.txt` file contained in a certain `G*/` folder (pick `G1/` for example):

```
./madevent < G1/input_app.txt
```

The code is instrumented with a timer, and at the end of this execution will show the total time spent in the computation of the Fortran overhead, of the matrix element, and the total time.

Exercise 2

Generate the `gg → t $\bar{t}$ gg` process, launch it once to verify the setup, and then run the `madevent` executable directly from the `SubProcesses/P1_gg_ttxgg` directory.

5 Exercise 3: Prepare for profiling

5.1 Add debug symbols and frame pointers

By default, the MadGraph build system does not provide debug symbols, nor frame pointers. Frame pointers are very useful when using `perf` to build a proper call stack, with the resulting flamegraphs showing less broken stacks or missing frames.

We need to modify two files.

5.1.1 Fortran compilation flags

Edit `Source/make_opts` (from the root of `MY_PROCESS`) and change the `GLOBAL_FLAG` line:

```
# Original:
GLOBAL_FLAG=-O

# Change to:
GLOBAL_FLAG=-O2 -g -fno-omit-frame-pointer
```

This affects all Fortran code (MadEvent framework, matrix elements in the Fortran version).

5.1.2 C++ and CUDA compilation flags

Edit `SubProcesses/P1_gg_ttxgg/cudacpp.mk` and modify the following lines.

Change CXXFLAGS:

```
# Original:
CXXFLAGS = $(OPTFLAGS) -std=c++17 -Wall -Wshadow -Wextra

# Change to:
CXXFLAGS = $(OPTFLAGS) -std=c++17 -Wall -Wshadow -Wextra -O2 -g -fno-omit-frame-pointer
```

Change OPTFLAGS:

```
# Original:
OPTFLAGS = -O3

# Change to:
OPTFLAGS = -O2 -g -fno-omit-frame-pointer
```

AMD GPU users

In case you are doing these exercises on an AMD GPU, there is an additional override of OPTFLAGS that should be updated:

```
# Original:
override OPTFLAGS = -O2

# Change to:
override OPTFLAGS = -O2 -g -fno-omit-frame-pointer
```

Are these flags passed also to the GPU builds?

In case of GPU builds, the main compiler, let's take `nvcc` (NVIDIA CUDA Compiler), does not support `-g -fno-omit-frame-pointer` flags (they are GCC flags). However, we may want to properly profile and show on a flamegraph the functions of the C++ files that are built with `nvcc` but that do not imply any GPU offloading. To do this, we can pass the `-g -fno-omit-frame-pointer` flags to the underlying compiler `g++`, used by `nvcc`, via `nvcc`'s `-Xcompiler` flags. This is done automatically already during build if the variable `OPTFLAGS` is correctly updated with those flags (as described above). The `OPTFLAGS` variable is used in the `nvcc` invocation.

5.2 Rebuild

Clean and rebuild everything. From the `SubProcesses/P1_gg_ttxgg/` directory:

```
make cleanall
make madevent_<backend>_link # build the C++ vectorised version
make madevent_fortran_link   # build also the Fortran-only version
```

Remember to substitute `<backend>` with whatever you chose in § 3.4.

This recipe `madevent_*_link` is present in the makefile to build at once all the files available in the different folders.

5.3 Increase the number of events

Profiling with a sampling profiler requires collecting enough samples to build statistically meaningful stack traces. At low sampling frequency, e.g. 97 Hz, a 1-second run yields only

≈97 samples, which may be too few for a meaningful flamegraph. We ideally need to collect thousands of samples.

Let's create a new input file `input_app.txt`, where we increase the number of events to at least **100 000** for CPU runs (the first integer in the first line), and **1 000 000** for GPU runs (in case you are trying these exercises on the GPU). The file would look like this (mind, also the two integer numbers next to the first integer changed as well):

```
cat > input_app.txt << EOF
100000 1 1 !Number of events and max and min iterations
0.1      !Accuracy
2        !Grid Adjustment 0=none, 2=adjust
1        !Suppress Amplitude 1=yes
0        !Helicity Sum/event 0=exact
1
EOF
```

Exercise 3

Modify the compilation flags as described, rebuild both the Fortran and vectorised versions of `madevent`, and increase the event count. Verify that both binaries run correctly and do not finish instantly.

6 Exercise 4: Flamegraphs

6.1 A brief reminder on flamegraphs

Flamegraphs are a visualisation of profiling data invented by Brendan Gregg. Each horizontal bar represents a function in the call stack. The **y-axis** is the stack depth (with entry point at the bottom, and *leaf* functions at the top). The **x-axis** is sorted alphabetically to merge identical stacks; the **width** of each bar is proportional to the number of samples in which that function appeared. Wider bars mean the function was on-CPU more often.

The workflow to produce a flamegraph is:

1. **record**: use `perf record` to collect stack samples;
2. **collapse**: convert the raw stack traces into a folded format;
3. **render**: generate an interactive SVG *picture*.

6.2 Profile the Fortran baseline

From the subprocess directory (`SubProcesses/P1_gg_ttxgg/`):

Record

```
perf record --call-graph=fp,1024 -F 97 -- ./madevent_fortran < input_app.txt
```

What does `--call-graph=fp,1024` do?

This option tells `perf` to use frame pointers (`fp`) to rebuild the stack of functions. The numeric parameter after the comma is the maximum stack depth. For this example, setting it to 1024 seems to be a good number.

Collapse

```
perf script | $FLAMEGRAPH_DIR/stackcollapse-perf.pl > fortran.folded
```

Render

```
$FLAMEGRAPH_DIR/flamegraph.pl fortran.folded > flamegraph_fortran.svg
```

What if it can't find stackcollapse-perf.pl?

This may happen because of two reasons:

- you did not download the Flamegraph repository as explained in § 3.3;
- you happened to have closed the current terminal, so now the environment variable \$FLAMEGRAPH_DIR is empty.

Have a look at § 3.3, and perform the installation or define the environment variable for the current session, if needed. For the latter, in case you downloaded the Flamegraph repository in your home directory, you can run the following:

```
export FLAMEGRAPH_DIR=$HOME/Flamegraph
```

Why 97 Hz and not 99 or 100?

We use a prime number for the sampling frequency to avoid aliasing with periodic patterns in the code. If the code has a loop that takes exactly 10 ms per iteration, and you sample at 100 Hz, every sample lands at the same point in the loop, and you don't want that, since it would introduce a bias in the profiling results. Prime frequencies avoid these synchronisation issues.

Open flamegraph_fortran.svg in a web browser (if you are working on a remote machine, remember to copy the SVG files to your laptop in order to view them in a browser, see § 2.1.1). The SVG is interactive: practice hovering over bars to see function names and sample percentages, click to zoom into a subtree, or search to highlight functions matching your search pattern (you can even use regular expressions).

Exercise 4a

Generate the flamegraph for the Fortran baseline. Identify the function that has been sampled the most, and note down its name, this is the current *bottleneck*, and it will be the function that is going to be hardware-accelerated. Write here the function name

. Hover over it and note the percentage of total samples it accounts for, and write down this number as well: %. From the standard output, you can also find the total wall-time, note it here as T_{Fortran} = s.

6.3 Profile the vectorised version

Now profile the vectorised C++ version (replace madevent_cpp with the appropriate binary name, in case you are trying these exercises on GPU):

```
perf record --call-graph=fp,1024 -F 97 -- ./madevent_cpp < input_app.txt
perf script | $FLAMEGRAPH_DIR/stackcollapse-perf.pl > cpp.folded
$FLAMEGRAPH_DIR/flamegraph.pl cpp.folded > flamegraph_cpp.svg
```

Exercise 4b

Generate the flamegraph for the vectorised version. Compare it visually with the Fortran flamegraph:

- What happened to the function that in the Fortran case was sampled the most?
- Which functions are now proportionally more visible?

Also in this case, note down the percentage of total samples that function now accounts for: %. Write also the total wall-time, T_{SIMD} = s.

6.4 Walking the stack: DWARF-based unwinding

An alternative stack-walking method, called *DWARF* is available. It uses debug unwind tables instead of frame pointers:

```
perf record --call-graph=dwarf -F 97 -- ./madevent_fortran < input_app.txt
```

This works even without frame pointers but produces larger `perf.data` files.

7 Exercise 5: Amdahl's law verification

Amdahl's law predicts the maximum overall speedup when only a fraction of the workload is accelerated:

$$S_{\text{Amdahl}} = \frac{1}{(1 - p) + \frac{p}{n}} \quad (1)$$

where p is the fraction of runtime that can be parallelised and n is the *number of processors* you are parallelising over.

7.1 Extract the data from the flamegraphs

From the wall-clock times of both runs, it is possible to compute:

- T_{Fortran} = s: total time for the Fortran run
- T_{SIMD} = s: total time for the vectorised run
- $S_{\text{observed}} = T_{\text{Fortran}} / T_{\text{SIMD}}$ = : the observed overall speedup

Additionally, from the Fortran flamegraph (Exercise 4a), you noted the percentage of time spent in the main bottleneck function, which is the function that has been accelerated in the hardware-accelerated version of the code.

Exercise 5

Using the flamegraph data and the wall-clock times:

1. Assign the values to p , s , n , according to the vectorisation level chosen:
 - $p =$
 - $s =$
 - $n =$
2. Compute S_{observed} from the wall-clock times.
 $S_{\text{observed}} =$
3. Verify Amdahl's law: does S_{Amdahl} , computed from eq. (1), match S_{observed} ?
 $S_{\text{Amdahl}} =$
4. Is the speedup smaller/larger than what you expect from the vectorisation level chosen? Why?
5. What would happen if we could parallelise over *infinitely* many processors ($n \rightarrow \infty$)? What is the theoretical maximum speedup? $S_{\infty} =$

8 Exercise 6: Hardware performance counters

In this section we are using another very useful function of perf, the `perf stat`, to study hardware counters.

8.1 General counters

Run the following for both the Fortran and vectorised versions:

```
perf stat -e task-clock,cycles,instructions,cache-misses,cache-references,L1-dcache-
load-misses \
-- ./madevent_fortran < input_app.txt
```

```
perf stat -e task-clock,cycles,instructions,cache-misses,cache-references,L1-dcache-
load-misses \
-- ./madevent_cpp < input_app.txt
```

Counter multiplexing

Your CPU has a limited number of hardware counter registers (typically 4–8). If you request more events than physical registers, perf time-shares them: each counter is active for only a fraction of the run, and the result is extrapolated. This is indicated by a percentage in parentheses, e.g. (55.56%), meaning the counter was measured for 55% of the runtime. For long, stable workloads, the extrapolation is reliable. If you need exact counts, split the measurements into separate runs with fewer counters each.

8.2 Floating-point width counters

These counters reveal *how* the CPU executes floating-point operations: scalar, 128-bit packed (SSE), 256-bit packed (AVX2), or 512-bit packed (AVX-512).

We also keep `task-clock` in the event list so that `perf stat` can compute human-readable derived metrics like operations per second, very handy to make comparisons. Additionally, `task-clock` is a software event and does not consume a hardware counter register.

In the following, you need to replace the placeholders `<FP_XXX>` with the appropriate counter names for your CPU. Indeed, counter names depend on vendor and microarchitecture, some examples for them are shown in the following table:

Width	Name example
Scalar	<code>fp_arith_inst_retired.scalar_double</code>
128-bit	<code>fp_arith_inst_retired.128b_packed_double</code>
256-bit	<code>fp_arith_inst_retired.256b_packed_double</code>
512-bit	<code>fp_arith_inst_retired.512b_packed_double</code>
Scalar	<code>fp_ops_retired_by_width.scalar_uops_retired</code>
128-bit	<code>fp_ops_retired_by_width.pack_128_uops_retired</code>
256-bit	<code>fp_ops_retired_by_width.pack_256_uops_retired</code>
512-bit	<code>fp_ops_retired_by_width.pack_512_uops_retired</code>

You can look them up on your machine by using `perf list` along with a keyword:

```
perf list floating
```

and then looking for keywords like `packed`, `sse`, `avx`, `retired`, ...In some cases, you will not have separate counters for different vectorisation levels, so you will just use one single counter that reports the operations with some levels of vectorisation.

```
perf stat -e task-clock,<FP_SCALAR>,<FP_128>,<FP_256>,<FP_512> \
-- ./madevent_fortran < input_app.txt
```

```
perf stat -e task-clock,<FP_SCALAR>,<FP_128>,<FP_256>,<FP_512> \
-- ./madevent_cpp < input_app.txt
```

8.3 How to interpret the results

Exercise 6

Collect the counters for both versions and fill in the following table (for the counters you have, you may not have some of them available on your system):

Metric	Fortran	Vectorised ()
Wall time (s)		
Instructions (B)		
Cycles (B)		
IPC (insn per cycle)		
Cache misses (M)		
Cache references (B)		
Cache miss rate (%)		
FP scalar ops (B)		
FP 128-bit packed ops (B)		
FP 256-bit packed ops (B)		
FP 512-bit packed ops (B)		

Then answer the following questions:

1. Look at the IPC, is it lower or higher in case of the vectorised version? Why?
2. Is the cache miss *rate* lower/higher for the vectorised version? Why?
3. What about the absolute number of cache misses?
4. Is there something weird happening in the FP width counters for the Fortran-only version? Remember, this is supposed to be scalar, with no vectorisation involved...
5. For the vectorised version, which width dominates? Is this consistent with the backend you chose?

The latter two questions work better in case you have separate counters for different levels of vectorisation.

9 Summary

In this session we covered the typical steps a simple and introductory profiling session may involve:

1. **Understand the code** by downloading MadGraph and doing some simple runs
2. **Instrument the build** for profiling (debug symbols, frame pointers).
3. **Record** stack samples with `perf record` and counters with `perf stat`.

4. **Visualise** call stacks with flamegraphs.
5. **Identify** the bottleneck and quantify its weight, and using Amdahl's law to predict the speedup.
6. **Iterate** once the bottleneck is addressed, the next one emerges.

As you saw, few commands are able to give you many insights about your code, guiding already several optimisations.

A key takeaway is that profiling is an iterative process. Profiling again at the end of any kind of optimisation work is essential to be able to understand what actually changed, if performances have improved, and whether new bottlenecks have appeared.

What about GPUs?

When the matrix element computation is offloaded to GPU, CPU-only profilers like `perf` can no longer see the offloaded computations. The CPU flamegraph will show time spent in CUDA runtime calls (e.g. `cudaDeviceSynchronize`) rather than in the code itself. To profile GPU workloads, tools like NVIDIA **Nsight Systems** (for CPU+GPU timeline visualisation) and **Nsight Compute** (for detailed GPU kernel analysis) are needed. These are covered in the GPU lecture.

10 References and useful links

- [1] Johan Alwall et al. "MadGraph 5 : Going Beyond". *JHEP* 06 (2011), p. 128. DOI: [10.1007/JHEP06\(2011\)128](https://doi.org/10.1007/JHEP06(2011)128). arXiv: [1106.0522](https://arxiv.org/abs/1106.0522) [hep-ph]
- [2] J. Alwall et al. "The automated computation of tree-level and next-to-leading order differential cross sections, and their matching to parton shower simulations". *JHEP* 07 (2014), p. 079. DOI: [10.1007/JHEP07\(2014\)079](https://doi.org/10.1007/JHEP07(2014)079). arXiv: [1405.0301](https://arxiv.org/abs/1405.0301) [hep-ph]
- [3] Stephan Hageböck et al. "Data-parallel leading-order event generation in MadGraph5_aMC@NLO" (07/2025). arXiv: [2507.21039](https://arxiv.org/abs/2507.21039) [hep-ph]
- [4] Andrea Valassi et al. "Madgraph on GPUs and vector CPUs: Towards production. The 5-year journey to the first LO release CUDACPP v1.00.00". *EPJ Web Conf.* 337 (2025), p. 01021. DOI: [10.1051/epjconf/202533701021](https://doi.org/10.1051/epjconf/202533701021). arXiv: [2503.21935](https://arxiv.org/abs/2503.21935) [physics.comp-ph]
- [5] MadGraph5_aMC@NLO collaboration. *MadGraph5_aMC@NLO source repository*. URL: <https://github.com/mg5amcnlo/mg5amcnlo> 16/3/2026
- [6] MadGraph4GPU collaboration. *MadGraph4GPU source repository*. URL: <https://github.com/madgraph5/madgraph4gpu> 16/3/2026
- [7] *perf wiki – Tutorial*. URL: <https://perf.wiki.kernel.org/index.php/Tutorial> 16/3/2026
- [8] Brendan Gregg. *Flame Graphs*. URL: <https://www.brendangregg.com/flamegraphs.html> 16/3/2026
- [9] Brendan Gregg. *FlameGraph tools*. URL: <https://github.com/brendangregg/FlameGraph> 16/3/2026